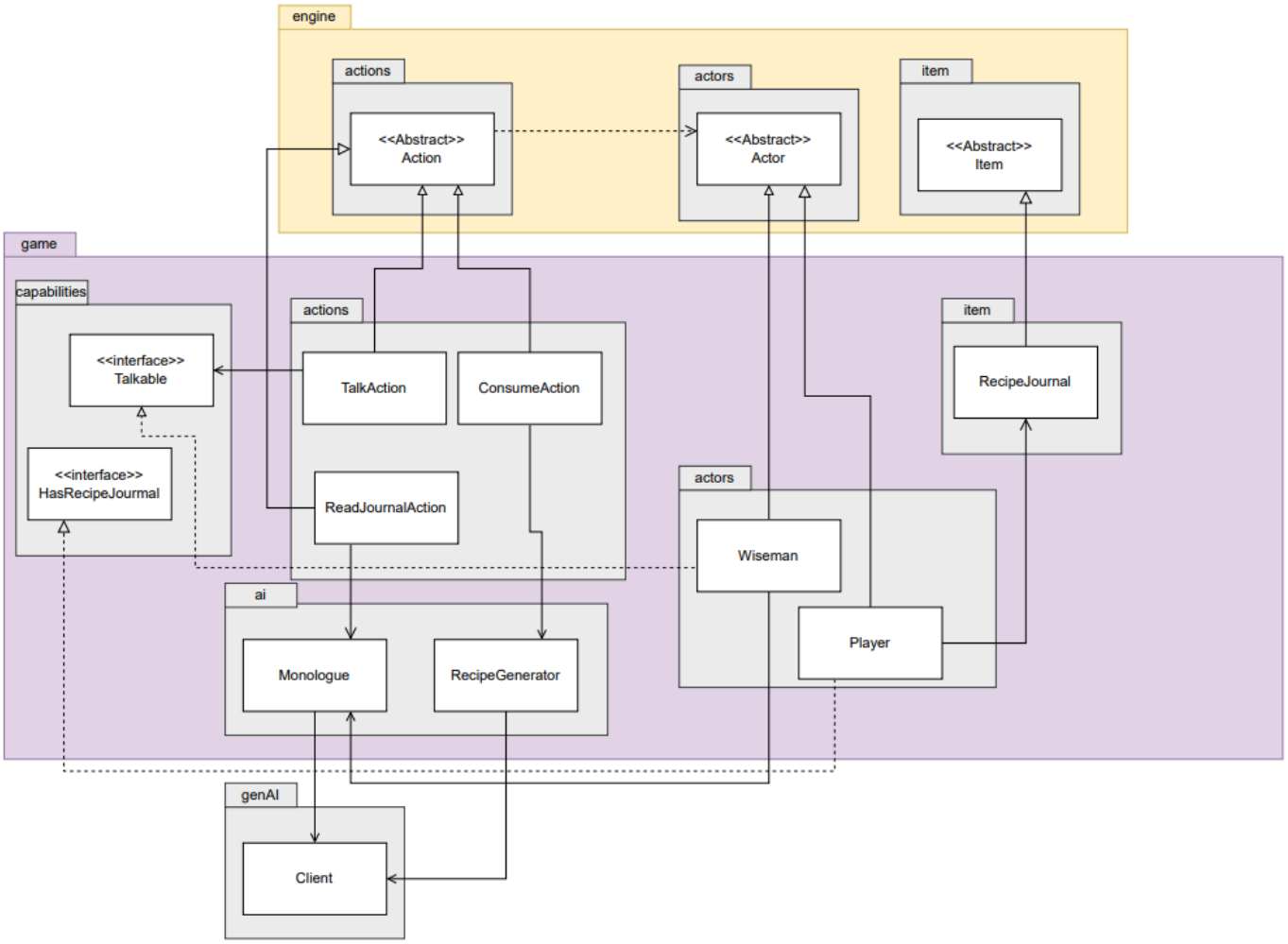
REQ 5 UML Class Diagram:



**Design Rationale**
REQ 5

A. Implementing WiseMan (NPC) actor speech behaviour
**Design 1:** Put all speech logic directly inside the WiseMan class. The WiseMan handles AI calls, context tracking, and deciding when to talk.

| | |
|---|---|
| Simple to implement: all functionality is in one place (KISS). | Violates Single Responsibility Principle (SOLID): WiseMan handles both actor behavior and AI speech logic. |
| No need for extra classes or interfaces. | Harder to test: tightly coupled logic with AI API cannot be easily mocked (SOLID – Dependency Inversion). |
| Direct access to WiseMan's attributes for context-specific speech. | Low flexibility: changing speech behavior requires modifying WiseMan class directly (Open/Closed Principle – SOLID). |
| Quick initial implementation for a single actor. | Not easily reusable: other actors cannot reuse the speech logic (DRY violation). |

**Design 2:** Create a Talkable interface implemented by WiseMan. The interface defines performMonologue(). When conditions are fulfilled (e.g., every 3 turns), a TalkAction is created and executed. A separate class Monologue is also created to call the API for response.

| Pros | Cons |
|---|---|
| Better abstraction: separates speech behavior from actor logic (Single Responsibility – SOLID). | Slightly more complex: introduces a new interface and action class |
| Testable: TalkAction and Talkable can be mocked in unit tests (Dependency Inversion – SOLID). | Requires careful design to manage turn counters or conditions outside WiseMan. |
| Reusable: other actors can implement Talkable and use the same TalkAction (DRY). | More initial setup needed (dependency injection or interface implementation). |
| Flexible: changing AI provider or behavior doesn't require modifying WiseMan directly (Open/Closed Principle – SOLID). | Minor overhead in managing TalkAction creation logic. |

B. Implementation of recipe generation and storing it

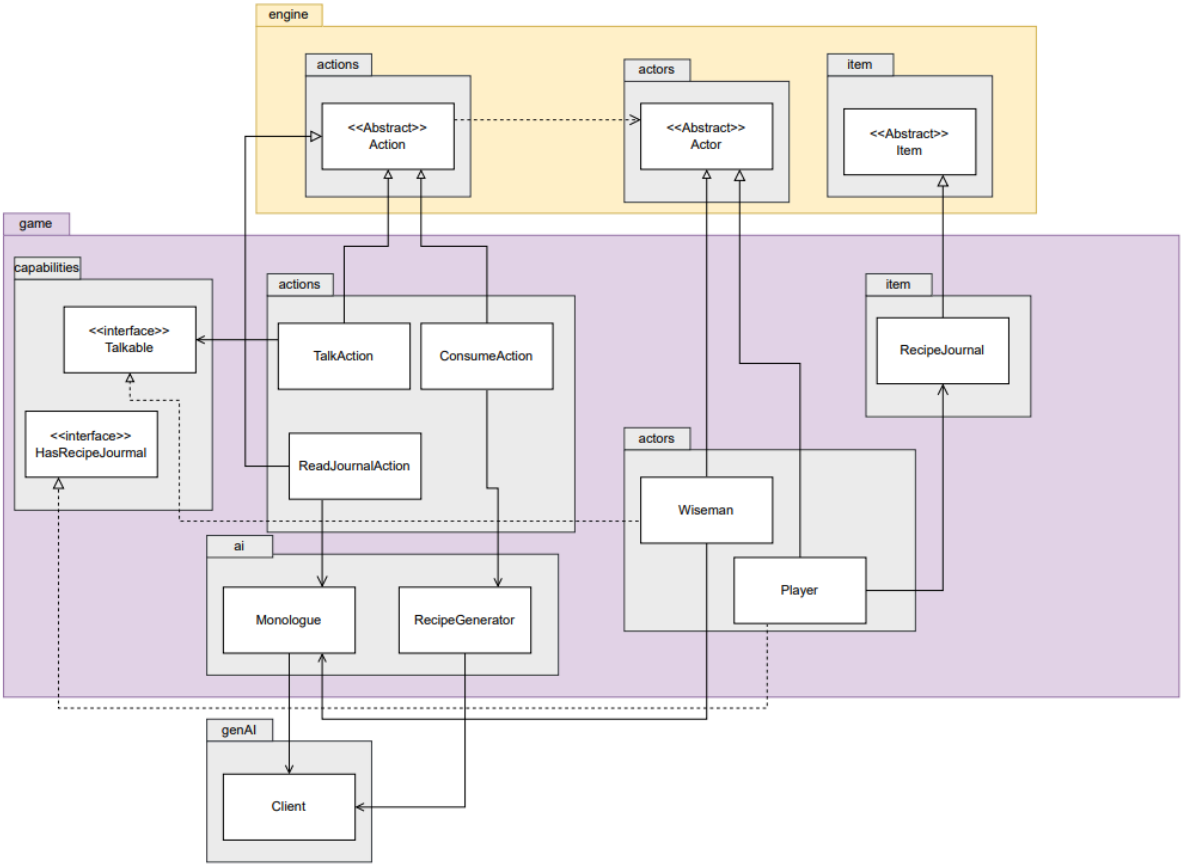
**Design 1:** The API call for generating recipes is directly combined inside the ConsumeAction class. ConsumeAction handles consumption, announcements, recipe generation and storing it all in one place.

| Pros | Cons |
|---|---|
| Simple to implement and easy to understand (KISS). | Violates Single Responsibility Principle (SOLID): ConsumeAction now handles both consuming items and recipe generation. |
| Minimal overhead: no extra classes needed. | Harder to test: tightly coupled with the AI API, cannot mock easily (Dependency Inversion violated – SOLID). |
| Quick for one actor type (player). | Violates DRY: recipe generation logic cannot be reused elsewhere. |
| Direct access to actor/item attributes. | Low flexibility: changing AI provider or recipe logic requires modifying ConsumeAction. |

.

**Design 2:** Create a separate RecipeGenerator class. ConsumeAction only checks if the actor is the player and, if so, records the recipe in a RecipeJournal which allows the player to use it and checks all the recipes stored.

| Pros | Cons |
|---|---|
| Follows Single Responsibility Principle (SOLID): ConsumeAction only consumes items; RecipeGenerator handles recipe generation. | Slightly more classes and setup (KISS slightly reduced). |
| Testable: RecipeGenerator can be mocked, avoiding actual API calls (Dependency Inversion – SOLID). | Minor additional setup needed to inject RecipeGenerator into ConsumeAction. |
| Reusable: recipe generation logic can be used elsewhere (DRY). | Slightly more complex architecture. |
| Flexible: changing AI provider or recipe behavior does not require modifying ConsumeAction (Open/Closed Principle – SOLID). | |




The diagram above shows the final design of requirement 5, which utilizes Design 2 of parts A and B. For the TalkAction, the Monologue class is responsible for interacting with the AI API and generating monologues, while the Talkable interface is implemented by the WiseMan. The WiseMan itself tracks turn-based conditions, and when the “every three turns” condition is fulfilled, it creates a TalkAction instance to perform speech. This separation ensures that WiseMan focuses only on character logic and turn tracking, while Monologue handles the AI communication. By using an interface and a dedicated action class, the design is reusable for other actors, follows the DRY principle, and supports testing via mocking of Monologue, adhering to the Dependency Inversion Principle (SOLID). The design also allows changes to the AI provider or speech logic without modifying WiseMan, satisfying the Open/Closed Principle, while remaining simple and readable in line with the KISS principle.

For recipe generation upon item consumption, the ConsumeAction class handles only item consumption, inventory or ground removal, and actor-specific announcements. The actual recipe generation is delegated to the RecipeGenerator class, which is injected into ConsumeAction and called only if the consuming actor is a player. This centralization reduces duplication (DRY) and keeps ConsumeAction focused (Single Responsibility – SOLID). Dependency injection of RecipeGenerator allows for easy testing and swapping of AI providers (Dependency Inversion – SOLID) and makes the system extensible for future features (Open/Closed Principle – SOLID). To store the recipe generated, it relies on RecipeJournal which uses ArrayList. This design remains straightforward and maintainable (KISS), while ensuring robust, reusable, and testable AI integration.

Overall, the combination of interfaces, dedicated action classes, and dependency injection results in a design that is extensible, maintainable, and aligned with core software engineering principles. Both speech and recipe generation are cleanly decoupled from actor logic, making the system scalable for additional AI-driven behaviors.